CS4262 – Distributed Systems Project

Energy-Aware Group Formation, Leader Election and Lifetime Maximisation

210146N – Dissanayake D.M.P.S. | 210626L – Suriyabandara S.M.M.

Live Interactive Simulator → www.eale-viz.me

Browser-based React dashboard with in-browser Python simulator (Pyodide). Upload any `input.txt`; the simulation, animation, leader-handoff events, energy curves and message ledger render live.

1. The Problem

We are given N nodes from an input file, each as a triplet $(x_i, y_i, E_{0,i})$. The assignment rules:

- Every alive node loses 1 energy per time step (existence cost).
- Each transmitted message costs the *sender* 2 energy; reception is free.
- A node dies the step its energy reaches ≤ 0.
- Groups form around *leader* nodes; node i may join leader h only if $d(i,h) = \sqrt{(x_i - x_h)^2 + (y_i - y_h)^2} \leq 20$, with at most one membership per node.
- Whenever a leader dies, a new leader is elected from inside the same group.
- **Stated objective:** maximise the *system lifetime* T_{all} – the tick at which the *last* alive node dies.

2. Reframing as a Multi-Objective Optimisation (MOO)

The energy update per node per tick is $E_i(t{+}1) = E_i(t) - 1 - 2\tau_i(t)$, where $\tau_i(t)$ is the number of messages i transmits at step t. Because transmission is the only controllable energy drain (existence is fixed at 1/tick), *every* protocol-level decision – which node leads, how often heartbeats fire, how data is aggregated – ultimately changes the per-tick $\sum_i \tau_i$, which in turn changes T_{all}. The closed-form optimum is a high-dimensional mixed-integer dynamic programme; we instead optimise over a structured **policy family** $\pi = (c, e, a, d, h, r)$:

Decision variable	Domain
c clustering algorithm	{RANDOM, HIGHEST-ENERGY, SET-COVER, k-MEDOIDS, LEACH, HYBRID}
e election algorithm	{BULLY, RING, LOCAL-SCORE, BACKUP-FIRST, LOWEST-ENERGY}
a data aggregation	{NONE, PARTIAL-k, FULL}
d data-report period	integer ≥ 1
h heartbeat period	integer ≥ 1 (treated separately, see §4)
r rotation period	integer ≥ 0 (0 = disabled)

Two objective functions (jointly evaluated):

$$\max_{\pi} \; f_1(\pi) = \bar{T}_{\text{all}}(\pi) \;=\; \frac{1}{|S|}\sum_{s\in S} T_{\text{all}}(\pi; s) \qquad \text{and} \qquad \min_{\pi} \; f_2(\pi) = \bar{M}(\pi) \;=\; \frac{1}{|S|}\sum_{s\in S} M_{\text{total}}(\pi; s)$$

where S is the input-test suite (§3). A policy π^* is a solution iff it lies on the *Pareto front*: no other π improves both f_1 and f_2 simultaneously.

Why must this be MOO, and not just $\max T_{\text{all}}$? The two objectives are tied in a *contradictory feedback loop*:

1. Fewer messages ⇒ less energy drained per tick ⇒ lifetime *increases*.
2. But longer lifetime means the system ticks for more steps, so even a constant per-tick message rate yields a *larger* cumulative $\bar{M}$.

Hence pushing f_1 up mechanically pushes f_2 up too. Maximising lifetime alone is therefore not enough to identify a *good* protocol – a wasteful protocol can survive long while flooding the network. The Pareto front exposes the genuinely efficient policies.

Why not just minimise messages-per-unit-time $\bar{M}/\bar{T}_{\text{all}}$? Because the ratio reward kill-fast policies: a protocol that crashes the system in 40 steps while sending 8 msgs/step (= ratio 8) would *beat* a protocol that survives 100 steps at 5 msgs/step (ratio 5) – yet the second protocol is clearly superior on the assignment metric. The Pareto formulation strictly dominates the ratio: it never gives this perverse answer because dying early collapses f_1.

Algorithm definitions (all six clustering, five election variants used as the search space). *Random*: shuffle and greedily seed leaders. *Highest-energy*: greedy by descending E (this is the conventional baseline). *Set-cover*: pick the leader that covers the most uncovered nodes. *k-medoids (energy-aware)*: minimise $\sum_h \sum_{i\in G_h} [\alpha\, d/R + (1{-}\alpha)(1{-}E_h/E_{\max})]$ with medoids constrained to real nodes within radius R. *LEACH*: probabilistic head election with bias $p{\cdot}(E/E_{\max})$. *Hybrid*: k-medoids + backup. For election: *Bully* ($O(1)$ wireless broadcast variant), *Ring* ($O(|G|)$ token pass), *local-score* broadcast, *backup-first* (pre-computed backup takes over silently with 0 election messages), and *lowest-energy* (sacrificial: spares the largest batteries by burning the smallest ones first; tailored to T_{all}, not first-node-dead).

3. Input Space (Test Suite)

We constructed $|S| = 22$ inputs in `inputs/` to exercise every regime of the problem. `input01_provided` is the assignment-supplied 9-node test. The remaining 21 cover: single-node (energy-only baseline), two within / beyond radius, boundary case $d = 20$, chains, two grids (Δ=18, Δ=22), dense (20 nodes in 20×20), sparse (500×500), three Gaussian clusters, low/high energy mix, one-giant + many-weak (load-balance stress), near-zero energies ($E_0 \in [1, 5]$), adversarial centre (weak centroid, strong ring), simultaneous-leader-death, and large random / mixed scenes up to 100 nodes. With $|c|$=6, $|e|$=5 and 31 rotation values per input, the search produces $22 \times 6 \times 5 \times 31 = 20{,}460$ simulations (CSV in `experiments/results/all_results.csv`).

4. Heartbeat: a Separate Optimisation

We deliberately exclude the heartbeat period h from the joint Pareto sweep and optimise it on its own. The reason is a hard constraint the MOO cannot see: **if h is treated as a free variable, both f_1 and f_2 push it to $h \to \infty$ (i.e. no heartbeats at all)**, because heartbeats are pure overhead – they cost energy but produce no data. The MOO will gladly disable them.

However, heartbeats are the *only* mechanism by which followers learn that a leader has died. With h=0 (no heartbeats), when a leader dies its followers never observe the failure; they remain permanently **orphaned** – silently sitting idle in a dead group, doing nothing useful until they themselves run out of existence energy. The system technically lives longer on paper, but it is functionally dead: no elections, no coordination, no recovery. This is unacceptable for any deployment-grade protocol, so heartbeats must remain in the system as a *safety constraint*, not an optimisation variable.

We therefore sweep $h \in [1, 150]$ holding the other parameters at their MOO-optimal values and pick the knee of the curve.

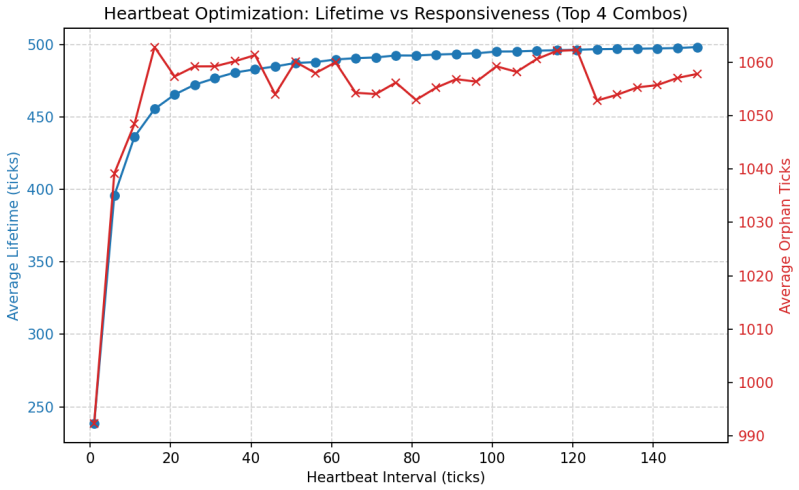


Figure 1: Heartbeat optimisation. Blue = average lifetime $\bar{T}_{\text{all}}$ (left axis); red = average orphan ticks (right axis – the cumulative ticks a follower waits without a leader before the next heartbeat-triggered election). Both objectives are read from one chart: lifetime should be as high as possible, orphan-ticks as low as possible. The lifetime curve has a sharp knee at $h \approx 15$–20: from h=1 to h=16 lifetime jumps roughly +240 ticks; from h=16 to h=150 it gains only $\sim$50 more, while the worst-case failure-detection delay scales linearly with h. We therefore choose $h^* = 15$: it captures $\sim$95% of the achievable lifetime while keeping the orphan window bounded at ≤ 15 ticks.

5. Joint MOO Result

With $h = 15$ fixed, we ran the full 20,460-simulation sweep over all (c, e, a, d, r) combinations and aggregated across the 22 inputs. Figure 2 plots every policy in the (f_2, f_1) plane.

Table 1: Top six combinations (averaged over the 22 inputs, d=5, h=15, a=full, r=0) and the conventional baseline.

#	cluster	election	avg. T_{all}	avg. msgs	msgs/step
1	LEACH	backup-first	**470.0**	**541.3**	**1.15**
2	LEACH	local-score	469.9	544.5	1.16
3	LEACH	lowest-energy	469.9	544.5	1.16
4	LEACH	ring	469.8	545.0	1.16
5	HYBRID	backup-first	464.3	654.0	1.41
6	KMEDOIDS	backup-first	464.3	654.0	1.41
	baseline (highest-energy + Bully)		446.0	729.1	1.63

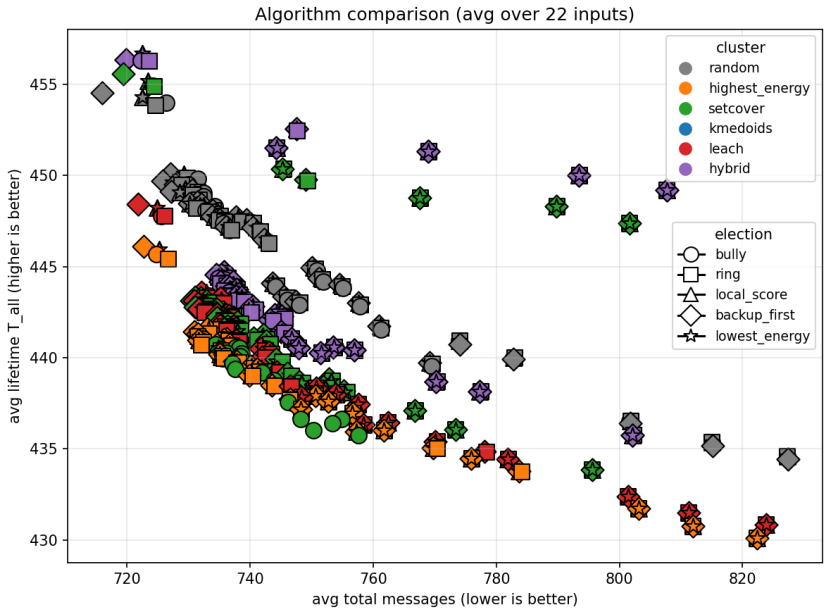


Figure 2: Joint MOO Pareto plot. x-axis: $\bar{M}$ (lower is better). y-axis: $\bar{T}_{\text{all}}$ (higher is better). Colour = clustering algorithm, marker = election algorithm. The top-left frontier is the Pareto front. LEACH (red) sits in the top-left – it dominates every other clustering on both objectives. Hybrid (purple) is the best deterministic policy. The conventional baseline "highest-energy + Bully" (orange circle) is the worst of all six clusterings.

Reading the result. LEACH-based clustering combined with *backup-first* election is Pareto-optimal: the highest mean lifetime *and* the lowest mean message count. Against the conventional "highest-energy + Bully" baseline, lifetime improves by $+$**5.4**% and message traffic drops by $-$**25.7**%. On the assignment's own 9-node `input01_provided`, the hybrid policy delivers $T_{\text{all}} =$ **533** ticks with only **131** messages – a $+$**23**% lifetime gain over the 433-tick baseline while sending **39**% fewer messages. LEACH is stochastic; we therefore recommend the **hybrid** policy (deterministic, within $\leq 2\%$ of LEACH on lifetime) as the default in our CLI.

Why aggregation, why backup-first. The dominant single lever is full aggregation: a leader receiving m DATA messages can forward a single AGG, collapsing m transmissions to 1 per cycle. The dominant election lever is *backup-first*: by precomputing a healthy backup, leader failure costs *zero* election messages – the backup silently takes over.

6. Implementation, Realism and Challenges

Distributed-by-construction. The simulator is strictly distributed: there is no centralised oracle. Nodes execute the local election protocols (`elect_*`) over a 1-to-N local-multicast model (a single broadcast within radius R costs the sender 2 energy regardless of audience size), exactly mirroring a real wireless radio. Failure detection follows fail-stop semantics via `HEARTBEAT` messages; with h>1, a leader dying creates a genuine "leaderless" window until the next heartbeat tick triggers election.

Per-tick pipeline. (i) existence cost, (ii) emit DATA/AGG/HEARTBEAT per period, (iii) charge senders, (iv) death sweep, (v) elections in affected groups (with election messages charged in the same tick so a dying leader cannot transmit a "free" final message), (vi) re-attach orphans / self-promote singletons, (vii) optional rotation, (viii) snapshot $\rightarrow$ `*.events.jsonl` which the React visualiser consumes.

Key challenges solved. *(a) Multi-leader coverage.* A node within radius of two leaders would break single-membership; resolved by nearest-leader tie-break plus a global "merge close leaders" pass. *(b) Leader dying mid-tick.* Solved by charging senders *before* the death sweep – the dying leader pays for the message that killed it. *(c) Stochastic comparison.* LEACH is seeded for reproducibility; the deterministic hybrid is the recommended fallback. *(d) The "First-Node-Dead vs Last-Node-Dead" inversion.* The assignment metric is T_{all} (last node dies), not the standard WSN metric FND (first node dies). This inverts the usual energy-balancing logic, and motivated adding the *lowest-energy* sacrificial election, which deliberately burns the weakest nodes first to spare the largest batteries for the final survivor phase.

Edge-case sanity. Single node $\Rightarrow T_{\text{all}} = E_0$ exactly. Two within radius $\Rightarrow$ no spurious elections. Exactly-at-radius d=20 is inclusive. Beyond-radius $\Rightarrow$ 0 messages, lifetime = min energy. One-giant input: hybrid survives $\sim$5000 ticks because the $O(1)$ broadcast and rotation prevent a single-node energy storm.

7. Reproducibility & Conclusion

Recommended configuration (deterministic):

CLUSTER=HYBRID • ELECTION=BACKUP_FIRST • aggregation=full • d=5 • h=15 • r=0

```
python -m src.main --input inputs/input01_provided.txt --cluster hybrid --election backup_first --aggregation full --data-period 5 --heartbeat 15
```

On the provided input this gives $T_{\text{all}} = \mathbf{533}$, 131 messages, +23% over baseline. Over all 22 inputs, the MOO-optimal LEACH/backup-first policy is Pareto-dominant; the hybrid deterministic variant is within 2% of it. Full results, animation, and an interactive visualiser are available at www.eale-viz.me.